CXL RAS Emulation Environment: Implementation & Setup Guide

Objective: Set up a comprehensive Compute Express Link (CXL) Reliability, Availability, and Serviceability (RAS) emulation utilizing Windows Subsystem for Linux (WSL2) and custom QEMU instances.
This guide provides instructions for injecting and detecting real-time CXL hardware error signals directly through the Linux kernel framework.

Environment Milestones:

- A custom-built WSL2 host environment (6.6.x kernel framework) with full QEMU KVM hardware virtualization support.
- A custom compiled QEMU 9.0.0 instance emulating two dual Type-3 CXL volatile memory devices.
- A standard upstream Linux 6.x development guest kernel that we compiled from source with native CXL RAS telemetry handlers explicitly enabled.
- Real-time diagnostic plumbing using the Linux kernel ftrace tracing engine to track driver responses to QMP fault payloads.

Core Concepts & Architectural Terms:

Windows (Host)
└── WSL2 (Ubuntu on Windows) — Our development environment
└── QEMU Process — emulates an x86 machine with CXL devices
└── Ubuntu Guest VM — runs a CXL-aware Linux kernel
└── CXL Driver — receives and processes error events

• **CXL (Compute Express Link):** A high-bandwidth, low-latency interconnect protocol designed to attach memory and accelerators directly to the CPU pipeline, replacing traditional motherboard RAM integration designs.

• **RAS (Reliability, Availability, and Serviceability):** The structural methodologies and engineering designs used to sense, handle, isolate and recover from hardware anomalies and device faults safely.

• **CCI (Component Command Interface):** The primary mailbox transport protocol utilized by the host operating system to interact with and control internal CXL device registries.

• **QMP (QEMU Machine Protocol):** A JSON-structured background management channel used to orchestrate, query and modify active hardware assets running inside a QEMU emulation layer.

• **Type-3 CXL Device:** A specialized CXL peripheral profile that acts as a dedicated memory expander target.

• **MSI-X (Message Signaled Interrupts Extended):** The specific PCI interrupt framework leveraged by advanced CXL endpoints to asynchronously notify the CPU for runtime events or error anomalies occur.

• **TCG vs. KVM Acceleration:** Tiny Code Generator (TCG) operates as a pure software emulation engine, which runs slowly and drops complex MSI-X interrupt sequences. Kernel-based Virtual Machine (KVM) leverages physical hardware extensions (Intel VT-x) to enable high performance and precise interrupt delivery.

Hardware & Environment Prerequisites:

- Windows 10 or 11 operating system with WSL2 architecture enabled.
- Minimum 16GB System RAM (Allocated as 8GB for the Windows Host layer and 8GB dedicated to the interior WSL2 + QEMU pipeline).
- At least 50GB of free unallocated storage space.
- An active WSL2 instance running an updated Ubuntu 24.04 LTS distribution baseline.

Part 1: Configuring the Custom WSL2 Host Kernel for KVM Support

By default, Microsoft WSL2 kernel configurations ship with KVM compiled as a module, but they do not include or unpack the compiled module packages into the distribution filesystem. We must compile the WSL2 kernel source to properly expose and mount the KVM hardware virtualization layer needed to handle guest MSI-X interrupts.

Step 1.1: Install Core Compilation Dependencies

```
sudo apt update
sudo apt install -y build-essential flex bison libssl-dev libelf-dev bc python3 pahole
```

Step 1.2: Fetch the Official WSL2 Kernel Source Tree

```
git clone https://github.com/microsoft/WSL2-Linux-Kernel.git --depth=1 -b linux-msft-wsl-6.6.y
cd WSL2-Linux-Kernel
```

Step 1.3: Import the Active Host Environment Configuration

```
zcat /proc/config.gz > .config
```

Step 1.4: Inject APEI EINJ Configuration Directives (Optional Baseline Testing)

```
scripts/config --enable CONFIG_ACPI_APEI_EINJ
make olddefconfig
```

Step 1.5: Compile the Custom Host Kernel

```
make -j$(nproc) KCONFIG_CONFIG=.config
```

Step 1.6: Transfer the Compiled Image to the Windows Storage Tier

Extract our exact Windows username via: 'ls /mnt/c/Users/' and copy the built image:

```
cp arch/x86/boot/bzImage /mnt/c/Users/<User>/wsl_kernel
```

Step 1.7: Link the WSL2 Subsystem to the New Kernel Image

Open Windows Notepad and create or modify the global configuration file located at: 'C:\Users\<User>\.wslconfig'

```
[wsl2]
kernel=C:\\Users\\<User>\\wsl_kernel
```

Step 1.8: Perform a Cold Restart of the WSL2 Subsystem

Executed the following lifecycle control commands inside a Windows PowerShell terminal session:

```
wsl –shutdown
wsl
```

Step 1.9: Initialize and Load the Host KVM Virtualization Modules

```
uname -r
cd ~/WSL2-Linux-Kernel
sudo make modules_install
```

```
sudo modprobe kvm
sudo modprobe kvm_intel

# Confirm successful kernel loading:
lsmod | grep kvm
```

Part 2: Compiling and Patching QEMU 9.0.0 from Source

Standard system packages provide QEMU 8.2.2, which contains an incomplete CXL layer. We deployed QEMU 9.0.0 and apply an intentional architectural patch to add the missing 'Set Partition Info' mailbox command handler.

Step 2.1: Install QEMU Source Compilation Tooling

```
sudo apt install -y git ninja-build python3-pip python3-venv libglib2.0-dev \
  libpixman-1-dev libslirp-dev libcap-ng-dev libattr1-dev \
  libsdl2-dev libgtk-3-dev libvte-2.91-dev libspice-server-dev \
  libaio-dev liburing-dev libseccomp-dev meson pkg-config
```

Step 2.2: Clone the QEMU Release Archive

```
mkdir -p ~/qemu
cd ~/qemu
git clone https://gitlab.com/qemu-project/qemu.git . --depth=1 -b v9.0.0
```

Step 2.3: Patch QEMU to Add the Set Partition Info Mailbox Command Handler

The Linux CXL driver utilizes the Set Partition Info command (opcode 0x4101) to structure hardware bounds. Since QEMU 9.0.0 does not implement this command, topology initialization fails. Open '~/qemu/hw/cxl/cxl-mailbox-utils.c' and apply the changes:

- A. Locate the declaration line: #define GET_PARTITION_INFO 0x0 and insert directly below it:

```
#define SET_PARTITION_INFO 0x1
```

- B. Append the execution code logic right after the 'cmd_ccls_get_partition_info' function scope.

```
/* CXL r3.1 Section 8.2.9.9.2.2: Set Partition Info (Opcode 4101h) */
static CXLRetCode cmd_ccls_set_partition_info(const struct cxl_cmd *cmd,
```

```
    uint8_t *payload_in, size_t len_in,
    uint8_t *payload_out, size_t *len_out, CXLCCI *cci)
{
    CXLDeviceState *cxl_dstate = &CXL_TYPE3(cci->d)->cxl_dstate;
    struct {
        uint64_t next_vmem;
        uint64_t next_pmem;
        uint8_t  flags;
    } QEMU_PACKED *set_pi = (void *)payload_in;
    if (len_in < sizeof(*set_pi)) return CXL_MBOX_INVALID_PAYLOAD_LENGTH;
    uint64_t next_vmem = ldq_le_p(&set_pi->next_vmem) * CXL_CAPACITY_MULTIPLIER;
    uint64_t next_pmem = ldq_le_p(&set_pi->next_pmem) * CXL_CAPACITY_MULTIPLIER;
    uint64_t total = cxl_dstate->vmem_size + cxl_dstate->pmem_size;
    if (next_vmem + next_pmem != total) return CXL_MBOX_INVALID_INPUT;
    cxl_dstate->vmem_size = next_vmem;
    cxl_dstate->pmem_size = next_pmem;
    return CXL_MBOX_SUCCESS;
}
```

- C. Bind the logic into the CCLS initialization table by placing this line immediately following the 'GET_PARTITION_INFO' configuration entry:

```
[CCLS][SET_PARTITION_INFO] = { "CCLS_SET_PARTITION_INFO",
cmd_ccls_set_partition_info, 0x11, IMMEDIATE_CONFIG_CHANGE },
```

Step 2.4: Configure and Build the Emulator Binary

```
cd ~/qemu
mkdir build && cd build
../configure --target-list=x86_64-softmmu --enable-slirp --enable-kvm
make -j$(nproc)
```

Step 2.5: Verify Emulator Engine Version

```
./qemu-system-x86_64 --version
# Confirm system outputs version 9.0.0
```

Part 3: Provisioning CXL Storage Backing Files and cloud-init Configurations

Step 3.1: Download the Base OS Image Vector

```
mkdir -p ~/cxl
cd ~/cxl
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
```

Step 3.2: Allocate Virtual CXL Disk Backing Space

```
dd if=/dev/zero of=~/cxl/cxlmem0.img bs=1M count=512
dd if=/dev/zero of=~/cxl/cxlmem1.img bs=1M count=512
```

Step 3.3: Build cloud-init Seed Structures for Automated Image Provisioning

```
cat > /tmp/user-data << 'EOF'
#cloud-config
users:
  - name: ubuntu
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    lock_passwd: false
    passwd: $6$rounds=4096$saltsalt$hashedpassword
chpasswd:
  list: |
    ubuntu:ubuntu
  expire: false
EOF

cat > /tmp/meta-data << 'EOF'
instance-id: cxl-test-01
local-hostname: ubuntu
EOF

sudo apt install -y cloud-image-utils
cloud-localds ~/cxl/seed.img /tmp/user-data /tmp/meta-data
```

Part 4: Compiling the Dedicated CXL Guest Kernel (Linux 7.0.0)

The standard cloud image kernel does not include crucial verification symbols required to test experimental CXL architectures. We deployed a custom Linux 7.0.0 system build using the development tree maintained directly by the upstream CXL kernel developers.

Step 4.1: Retrieve the Upstream CXL Subsystem Tree

```
cd ~/cxl
git clone https://git.kernel.org/pub/scm/linux/kernel/git/cxl/cxl.git linux --depth=1
cd linux
```

Step 4.2: Set Up Base Kernel Compilation Profiles

```
cp /boot/config-$(uname -r) .config 2>/dev/null || zcat /proc/config.gz > .config
make olddefconfig
```

Step 4.3: Toggle Explicit Verification and Testing Kernel Parameter Symbols

```
scripts/config --enable CONFIG_CXL_BUS
scripts/config --enable CONFIG_CXL_PCI
scripts/config --enable CONFIG_CXL_MEM
scripts/config --enable CONFIG_CXL_PORT
scripts/config --enable CONFIG_CXL_ACPI
scripts/config --enable CONFIG_CXL_REGION
scripts/config --enable CONFIG_CXL_REGION_INVALIDATION_TEST
scripts/config --enable CONFIG_CXL_FEATURES
scripts/config --enable CONFIG_CXL_ATL
scripts/config --enable CONFIG_MEMORY_FAILURE
scripts/config --enable CONFIG_ACPI_APEI
scripts/config --enable CONFIG_ACPI_APEI_GHES
scripts/config --enable CONFIG_RAS
make olddefconfig
```

• Architectural Note on CONFIG_CXL_REGION_INVALIDATION_TEST: Emulation architectures (TCG/KVM) cannot invoke real CPU cache invalidation instructions natively. This test parameter forces the driver to skip the low-level hardware verification step, allowing virtualized region configurations to succeed.

• Architectural Note on CONFIG_MEMORY_FAILURE: This serves as a vital gatekeeper variable enabling it forces the kernel configuration script to activate the downstream CONFIG_CXL_MCE architecture, routing error vectors cleanly into the active system trace infrastructure.

Step 4.4: Execute Guest Kernel Code Compilation

```
make -j$(nproc) 2>&1 | tail -5
```

Step 4.5: Transfer the Built Image to Target Windows Mount Space

```
cp arch/x86/boot/bzImage /mnt/c/Users/<User>/cxl_guest_kernel
```

Part 5: Creating the Comprehensive QEMU Machine Deployment Script

Write the core automated initialization script to '~/cxl/start-cxl.sh':

```
cat > ~/cxl/start-cxl.sh << 'EOF'
#!/bin/bash
/home/user/qemu/build/qemu-system-x86_64 \
  -kernel /mnt/c/Users/<User>/cxl_guest_kernel \
  -append "root=/dev/vda1 console=ttyS0 console=tty1" \
  -m 2G -smp 4 \
  -machine q35,cxl=on,cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=4G \
  -cpu qemu64 -nographic -serial mon:stdio \
  -accel kvm \
  -D /tmp/qemu.log -d guest_errors,trace:cxl* \
  -object memory-backend-file,id=mem0,mem-
path=/home/user/cxl/cxlmem0.img,size=512M,share=on \
  -object memory-backend-file,id=mem1,mem-
path=/home/user/cxl/cxlmem1.img,size=512M,share=on \
  -device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1 \
  -device cxl-rp,port=0,bus=cxl.1,id=rp0,chassis=0,slot=2 \
  -device cxl-rp,port=1,bus=cxl.1,id=rp1,chassis=0,slot=3 \
  -device cxl-type3,bus=rp0,volatile-memdev=mem0,id=cxl0,sn=1 \
  -device cxl-type3,bus=rp1,volatile-memdev=mem1,id=cxl1,sn=2 \
  -drive file=/home/user/cxl/noble-server-cloudimg-amd64.img,if=none,id=hd0,format=qcow2 \
  -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
  -drive file=/home/user/cxl/seed.img,if=none,id=seed0,format=raw \
  -device virtio-blk-pci,drive=seed0,bus=pcie.0 \
  -qmp tcp:127.0.0.1:4444,server,nowait
EOF
chmod +x ~/cxl/start-cxl.sh
```

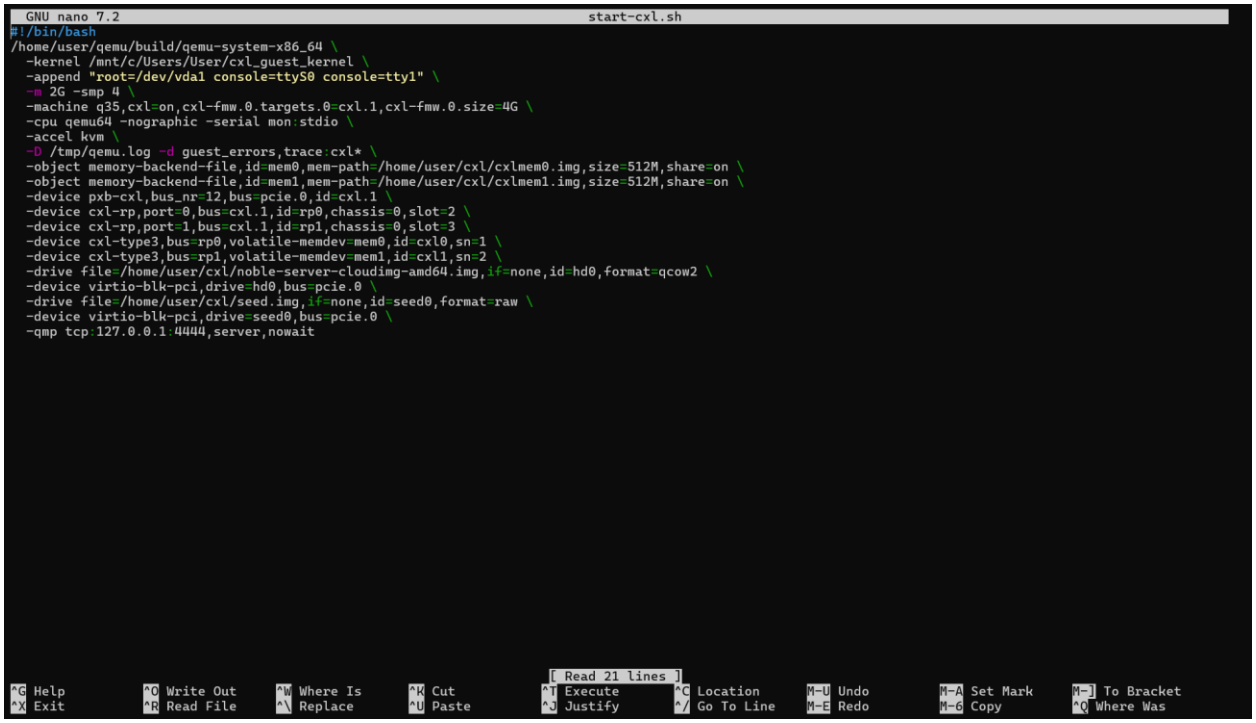


figure showing output of nano start-cxl.sh

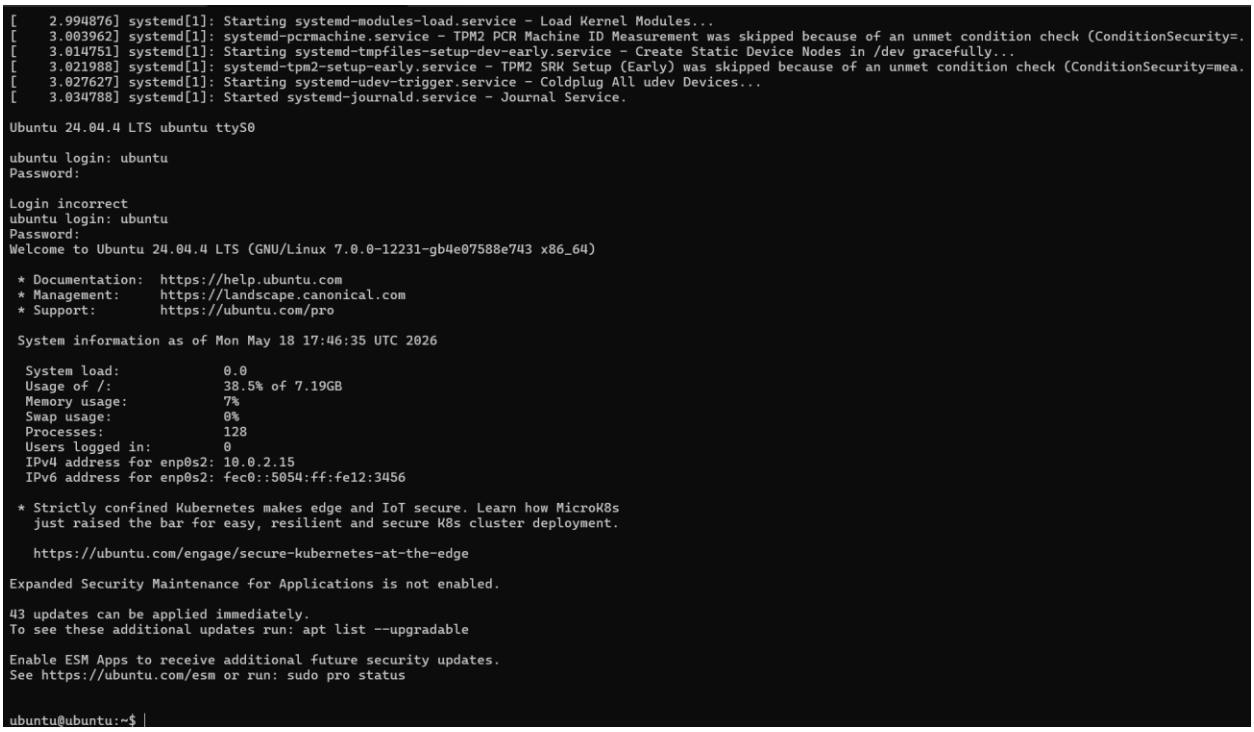
Key Architecture Launch Flags Explained:

- • '-accel kvm': Invokes hardware virtualization acceleration; mandatory for routing MSI-X device error interrupts smoothly into the guest kernel runtime.
- • '-kernel / -append': Boots the virtual machine directly using our newly built Linux 7.0.0 image, bypassing the internal disk bootloader layout.
- • 'cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=4G': Allocates a system-level 4GB CXL Fixed Memory Window block.
- • 'cxl-type3,sn=1': Directs QEMU to append tracking serial numbers to the endpoints, a structural requirement for correct device registration in QEMU 9.0.
- • '-qmp tcp:127.0.0.1:4444': Opens a local TCP network listener to ingest structural JSON hardware fault injection payloads.

Part 6: Guest Environment Initialization & Package Locking

Step 6.1: Spin Up the Virtual Machine Engine Instance

```
cd ~/cxl && ./start-cxl.sh
```



Result of the above command

Step 6.2: Complete Internal Operating System Setup

Log in using the credential profiles configured in the seed metadata files (ubuntu / ubuntu) and initialize the management tooling:

```
sudo apt update
sudo apt install -y linux-modules-extra-$(uname -r) ndctl
```

Step 6.3: Apply Explicit Package Image Version Locks

Lock down the active kernel files to shield our environmental runtime from automated background updates:

```
sudo apt-mark hold linux-image-$(uname -r) linux-modules-$(uname -r) linux-modules-extra-$(uname -r)
```

Part 7: Live Hardware Error Injection and Diagnostic RAS Monitoring

Follow this precise workflow sequence whenever executing error injection testing scenarios:

Step 7.1: Expose Host Extensions and Launch QEMU (Host Terminal Window)

```
sudo modprobe kvm
sudo modprobe kvm_intel
cd ~/cxl && ./start-cxl.sh
```

Step 7.2: Instantiate and Map a CXL Memory Region (Inside Guest Console Window)

```
sudo su
cxl create-region -d decoder0.0 -m mem1 -s 512M -t ram
```

The orchestrator utility will confirm successful generation by outputting the active system memory allocation mapping:

```
{
  "region":"region0",
  "resource":"0x110000000",
  "size":"512.00 MiB (536.87 MB)",
  "type":"ram",
  "decode_state":"commit"
}
cxl region: cmd_create_region: created 1 region
```

Step 7.3: Open the Kernel Tracing Subsystem (Inside Guest Console Window)

```
echo 1 > /sys/kernel/debug/tracing/events/cxl/enable
echo 1 > /sys/kernel/debug/tracing/tracing_on
cat /sys/kernel/debug/tracing/trace_pipe &
```

Step 7.4: Initialize a Remote QMP Handshake Session (Open a New Separate Host Terminal Window)

```
nc 127.0.0.1 4444

# Negotiate initial session handshake capabilities:
{"execute": "qmp_capabilities"}
```

Step 7.5: Inject a General Media Fault Event Profile via QMP

Issue the target JSON fault payload over the active QMP network terminal socket. Note that the target coordinate '4563402752' represents the base decimal equivalent of the system mapping memory address (0x110000000):

```
{"execute": "cxl-inject-general-media-event", "arguments": {"path": "/machine/peripheral/cxl1", "type": 0, "flags": 0, "dpa": 4563402752, "dpa-length": 64}}
```

Figure showing the injection of General Media Fault via QMP

Analyze the simultaneous guest console output; the ftrace log channel will catch the incoming interrupt assertion:

```
irq/40-0000:0e:-77 [003] 324.455313: cxl_general_media: memdev=mem0
host=0000:0e:00.0 serial=2 log=Informational
```

```
[    3.177131] systemd[1]: Starting modprobe@fuse.service - Load Kernel Module fuse...
[    3.187742] systemd[1]: Starting modprobe@loop.service - Load Kernel Module loop...
[    3.195135] systemd[1]: netplan-ovs-cleanup.service - OpenVSwitch configuration for cleanup was skipped because of an unmet condition check (ConditionFi.
[    3.203194] systemd[1]: Starting systemd-fsck-root.service - File System Check on Root Device...
[    3.216506] systemd[1]: Starting systemd-modules-load.service - Load Kernel Modules...
[    3.221133] systemd[1]: systemd-pcrmachine.service - TPM2 PCR Machine ID Measurement was skipped because of an unmet condition check (ConditionSecurity=.
[    3.226710] systemd[1]: Starting systemd-tmpfiles-setup-dev-early.service - Create Static Device Nodes in /dev gracefully...
[    3.236944] systemd[1]: systemd-tpm2-setup-early.service - TPM2 SRK Setup (Early) was skipped because of an unmet condition check (ConditionSecurity=mea.
[    3.245948] systemd[1]: Starting systemd-udev-trigger.service - Coldplug All udev Devices...
[    3.261363] systemd[1]: Started systemd-journald.service - Journal Service.

Ubuntu 24.04.4 LTS ubuntu ttyS0

ubuntu login: ubuntu
Password:

Login incorrect
ubuntu login: ubunut
Password:

Login incorrect
ubuntu login: ubuntu
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-12231-gb4e07588e743 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon May 18 15:33:25 UTC 2026

  System load:             0.0
  Usage of /:              38.4% of 7.19GB
  Memory usage:            5%
  Swap usage:              0%
  Processes:               113
  Users logged in:         0
  IPv4 address for enp0s2: 10.0.2.15
  IPv6 address for enp0s2: fec0::5054:ff:fe12:3456

root@ubuntu:/home/ubuntu# echo 1 > /sys/kernel/debug/tracing/events/cxl/enable
root@ubuntu:/home/ubuntu# echo 1 > /sys/kernel/debug/tracing/tracing_on
root@ubuntu:/home/ubuntu# cat /sys/kernel/debug/tracing/trace_pipe
 irq/44-0000:0e:-83      [003] .....   937.980117: cxl_general_media: memdev=mem1 host=0000:0e:00.0 serial=2 log=Informational : time=1779118746157451874 u0
```

Figure showing the Detection of General Media Error

Step 7.6: Inject a Volatile DRAM Fault Event Profile via QMP

```
{"execute": "cxl-inject-dram-event", "arguments": {"path": "/machine/peripheral/cxl1", "type": 0, "flags": 0, "dpa": 4563402752, "dpa-length": 64}}
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\User> wsl
user@DESKTOP-MORQBDS:/mnt/c/Users/User$ nc 127.0.0.1 4444
{"QMP": {"version": {"qemu": {"micro": 0, "minor": 0, "major": 9}, "package": "v9.0.0-dirty"}, "capabilities": ["oob"]}}
{"execute": "qmp_capabilities"}
{"return": {}}
{"execute": "cxl-inject-dram-event", "arguments": {"path": "/machine/peripheral/cxl1", "type": 0, "flags": 0, "dpa": 4563402752, "descriptor": 0, "log": "in
formational", "transaction-type": 0, "channel": 1, "rank": 2, "nibble-mask": 0, "bank-group": 1, "bank": 2, "row": 1024, "column": 512, "correction-mask": [
0, 0, 0, 0]}}
{"return": {}}
{"execute": "cxl-inject-dram-event", "arguments": {"path": "/machine/peripheral/cxl1", "type": 0, "flags": 0, "dpa": 4563402752, "descriptor": 0, "log": "in
formational", "transaction-type": 0, "channel": 1, "rank": 2, "nibble-mask": 0, "bank-group": 1, "bank": 2, "row": 1024, "column": 512, "correction-mask": [
0, 0, 0, 0]}}
{"return": {}}
{"return": {}}
{"error": {"class": "GenericError", "desc": "QMP input member 'return' is unexpected"}}
{"timestamp": {"seconds": 1779119166, "microseconds": 384569}, "event": "RTC_CHANGE", "data": {"offset": 0, "qom-path": "/machine/unattached/device[6]/rtc"}
}
```

Figure showing the injection of DRAM Fault via QMP

The real-time tracing stream will output the decoded hardware fault attributes caught by the active handler:

irq/40-0000:0e:-77 [003] 398.408547: cxl_dram: memdev=mem0 host=0000:0e:00.0 serial=2 log=Informational : time=... uuid=...

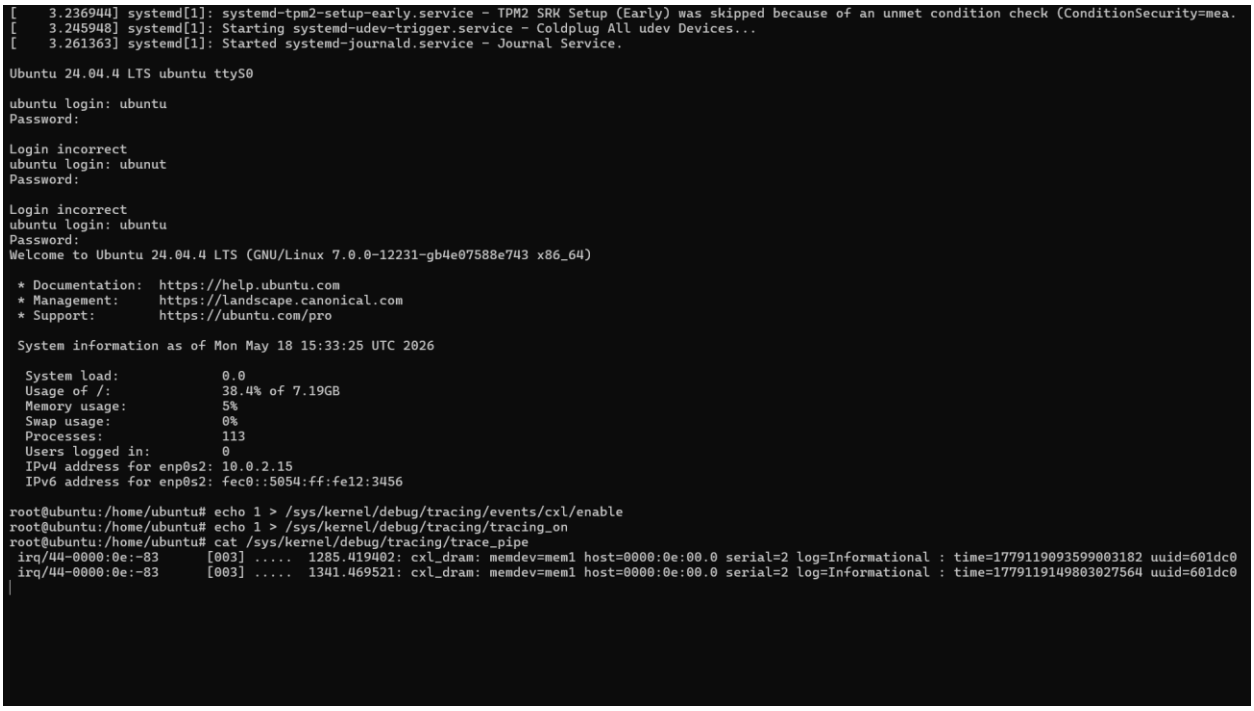


Figure showing the Detection of DRAM Fault

Part 8: Hardware Trace Packet & Signal Path Anatomy

1. **QMP Injection (Host Level):** A raw JSON error payload is sent from the host via network socket (nc 127.0.0.1 4444) to QEMU's Component Command Interface (CCI).

2. **Device Emulation (Hardware Level**): QEMU processes the payload, emulates physical register state adjustments, and caches the error data within the virtual CXL device's internal event log banks.

3. **MSI-X Interrupt (Bus Level**): The emulated CXL device asserts a hardware-level interrupt up to the CPU. In our specific environment, this dynamically flags IRQ 44 on the physical footprint layout (0000:0e:00.0).

4. **Driver Handling (Kernel Level**): The Linux Operating System intercepts the interrupt, recognizes it belongs to the CXL subsystem, and hands execution control to the core CXL drivers.

5. **CCI Mailbox Read (Interface Level):** The active CXL driver issues a command back down the virtual system bus to open the device's CCI mailbox and extract the raw error telemetry.

6. **ftrace Logging (User/Debug Space):** The driver parses the raw bits into structured text, writing it to the kernel's tracking ring buffer. This makes it instantly readable via /sys/kernel/debug/tracing/trace_pipe.

Deconstructed ftrace Fields

For DRAM Event

- **irq/44-0000:0e:-83 (Task/PID):** The dedicated kernel service thread handling the hardware IRQ execution path.

- **[003] (CPU Core):** The specific virtual CPU core executing the driver sequence when the interrupt landed.

- **1285.419402 (Timestamp):** Precise system uptime clock counter, in seconds, measured since the Guest VM booted.

- **cxl_dram: (Event Name):** The core driver trace point class identifier. *(Swaps to cxl_general_media: when running media-specific validations)*.

- **memdev=mem1 host=0000:0e:00.0 (Metadata):** Explicit system hardware mapping attributes pointing directly to our active CXL memory expander and its physical PCIe bus address location.

Available Injection Commands

QMP Command	What It Simulates
cxl-inject-general-media-event	General media error at a memory address
cxl-inject-dram-event	DRAM-level fault (row/column/bank specific)
cxl-inject-memory-module-event	Module-level failure event
cxl-inject-correctable-error	Correctable hardware error (device recovers)
cxl-inject-uncorrectable-errors	Uncorrectable error (may take device offline)

Environment Summary

Component	Version / Detail
WSL2 Kernel	6.6.123.2-microsoft-standard-WSL2+ (custom, KVM modules installed)
QEMU	9.0.0 (built from source, patched with Set Partition Info)
Guest OS	Ubuntu 24.04 (noble-server-cloudimg)
Guest Kernel	Linux 7.0.0 (CXL development kernel)
CXL Devices	2x Type-3 volatile memory, 512MB each
Interrupt Delivery	MSI-X via KVM
Error Injection	QMP over TCP port 4444

Component	Version / Detail
Event Observation	Linux kernel tracing subsystem (trace_pipe)